

Compte-Rendu_TP6

Création VM:

RT-box(interne/brigde) ou RT-box(interne/NAT) on s'en fou en vrai
VM ubuntu desktop (interne / mac aléatoire)

installation wireshark:

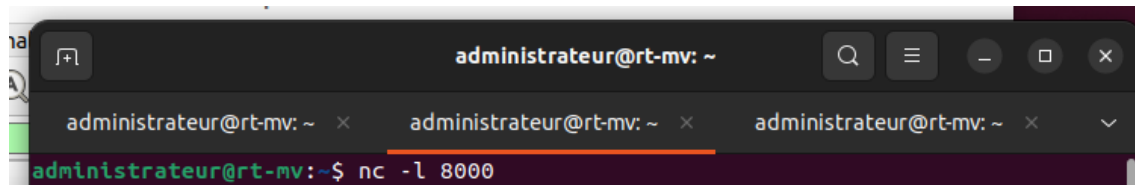
sudo apt update

sudo apt install wireshark (mettre oui au message bizarre en rose)

dans un terminal mettre: sudo wireshark (avoir les permissions admin sur wireshark)

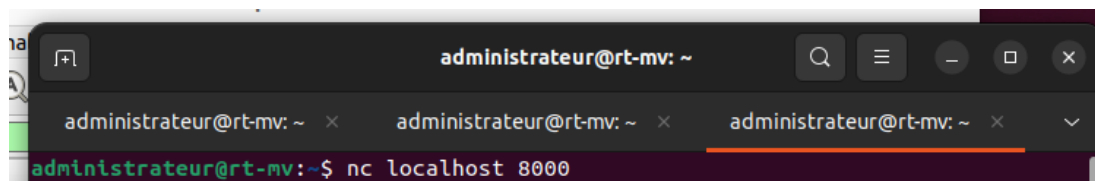
lancer wireshark s'il ne se lance pas tout seul et choisir **loopback** et dans filtre tcp

dans un nouveau terminal mettre la commande: `nc -l 8000`



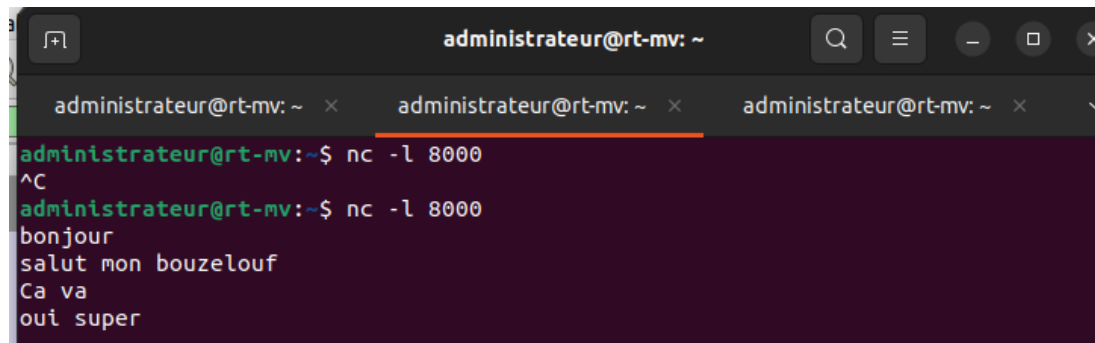
```
administrateur@rt-mv: ~  
administrateur@rt-mv: ~$ nc -l 8000
```

dans un autre terminal mettre la commande: `nc localhost 8000`

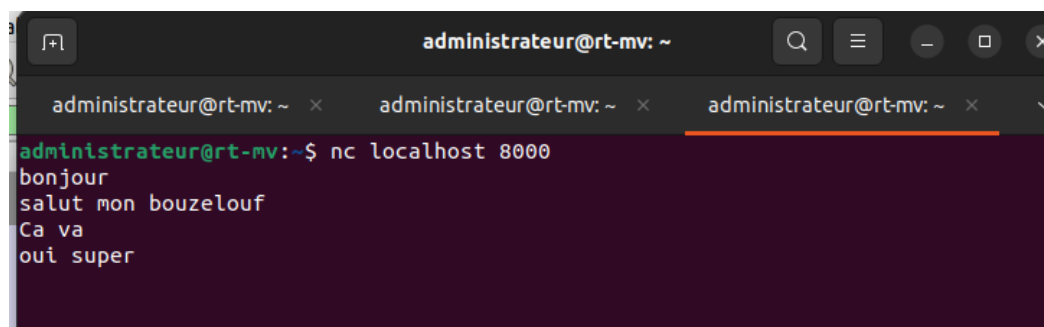


```
administrateur@rt-mv: ~  
administrateur@rt-mv: ~$ nc localhost 8000
```

apres ecrire dans le terminal chaque bon message dans le bon terminal



```
administrateur@rt-mv: ~  
administrateur@rt-mv: ~$ nc -l 8000  
^C  
administrateur@rt-mv: ~$ nc -l 8000  
bonjour  
salut mon bouzelouf  
Ca va  
oui super
```



```
administrateur@rt-mv: ~  
administrateur@rt-mv: ~$ nc localhost 8000  
bonjour  
salut mon bouzelouf  
Ca va  
oui super
```

après avoir écrit le texte faire CTRL D

couper la capture wireshark

En faisant un **clik droit** sur une trame de cet échange puis **"follow ou suivre" | "TCP stream"** (suivre ce flux TCP), vous pouvez voir par exemple une capture du flux TCP associé à l'exemple précédent.



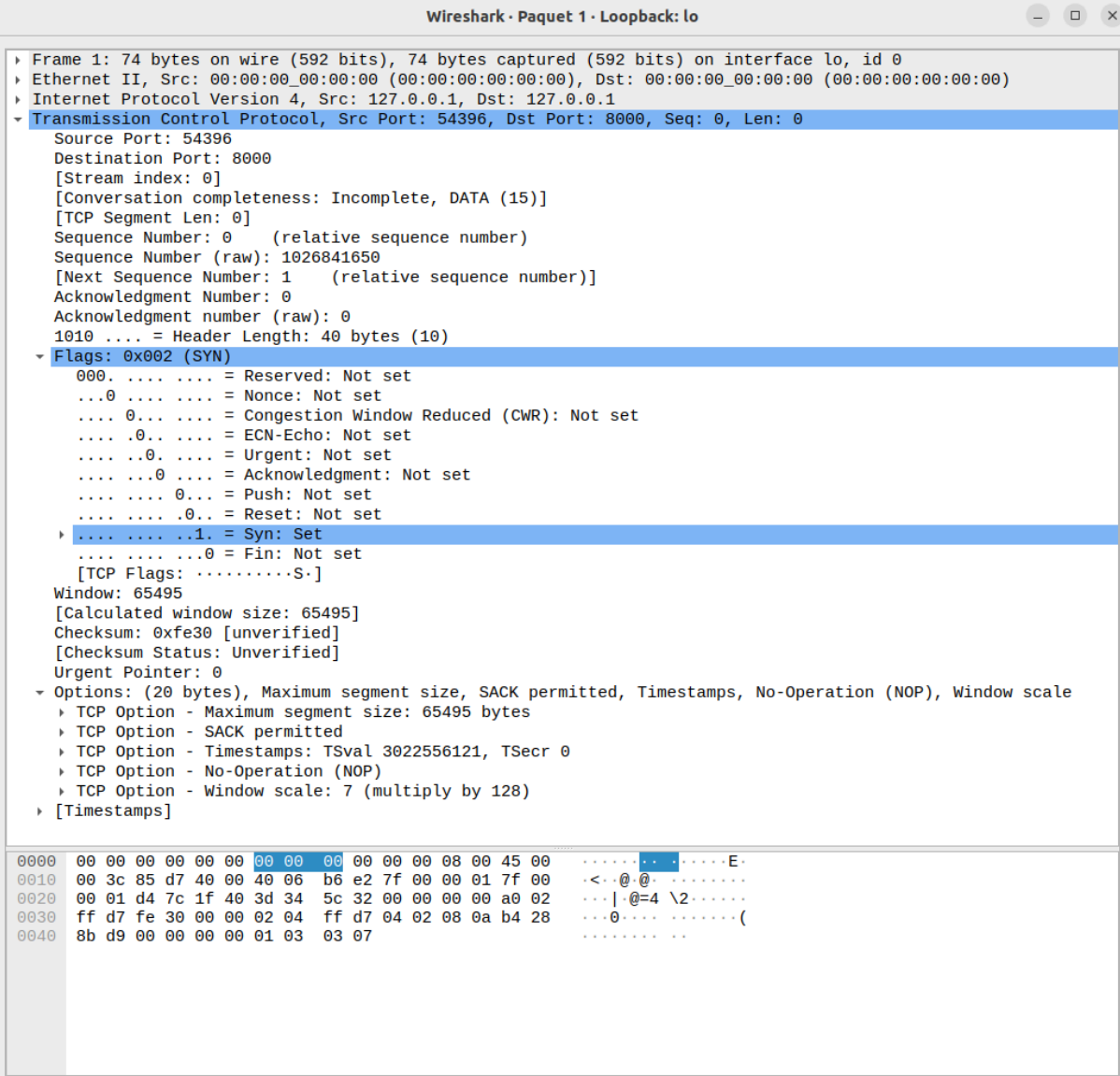
Dans le menu **"statistics"**, vous avez également **"Flow graph"** pour voir l'illustration de l'échange des trames. Spécifiez **"TCP flows"** (flux TCP) dans Flow type et **"Limit to display filter"** pour avoir que le trafic TCP concerné, et voir par exemple un graphe d'échanges TCP comme le suivant.

Temps	127.0.0.1	Commentaire
0.000000000	8000 → 51132	Seq = 0
0.000010147	51132 → 8000	Seq = 0 Ack = 1
0.000017419	8000 → 51132	Seq = 1 Ack = 1
60.145688068	51132 → 8000	Seq = 1 Ack = 1
60.145710383	8000 → 51132	Seq = 1 Ack = 9
69.558938024	8000 → 51132	Seq = 1 Ack = 9
69.558958722	51132 → 8000	Seq = 9 Ack = 21
79.187827744	51132 → 8000	Seq = 9 Ack = 21
79.187850098	8000 → 51132	Seq = 21 Ack = 15
88.416282438	8000 → 51132	Seq = 21 Ack = 15
88.416304778	51132 → 8000	Seq = 15 Ack = 31

Vérifiez que les numéros de séquence et d'acquittement correspondent bien à ce que vous avez vu en **cours** et TD. Par exemple : **numéro de séquence (Seq) + longueur des données (Len) = numéro d'acquittement (Ack)**

Vérification Seq + Len = Ack — par exemple **Bonjour** = 7 caractères + **\n** = 8 octets → Seq=1 + 8 = Ack=9.

Le champ taille de fenêtre (Win) est codé sur 2 octets, sa valeur maximale est donc de 65535 octets. Pourtant vous pouvez apercevoir des valeurs plus grande pour ce champ dans cet échange. Vous pouvez avoir cela par exemple :



```
Wireshark - Paquet 1 - Loopback: lo
Frame 1: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface lo, id 0
Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 54396, Dst Port: 8000, Seq: 0, Len: 0
  Source Port: 54396
  Destination Port: 8000
  [Stream index: 0]
  [Conversation completeness: Incomplete, DATA (15)]
  [TCP Segment Len: 0]
  Sequence Number: 0 (relative sequence number)
  Sequence Number (raw): 1026841650
  [Next Sequence Number: 1 (relative sequence number)]
  Acknowledgment Number: 0
  Acknowledgment number (raw): 0
  1010 .... = Header Length: 40 bytes (10)
  Flags: 0x002 (SYN)
    000. .... = Reserved: Not set
    ...0 .... = Nonce: Not set
    .... 0... = Congestion Window Reduced (CWR): Not set
    .... 0... = ECN-Echo: Not set
    .... 0... = Urgent: Not set
    .... 0... = Acknowledgment: Not set
    .... 0... = Push: Not set
    .... 0... = Reset: Not set
    .... 0... = Syn: Set
    .... 0... = Fin: Not set
    [TCP Flags: .....S.]
  Window: 65495
  [Calculated window size: 65495]
  Checksum: 0xfe30 [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  Options: (20 bytes), Maximum segment size, SACK permitted, Timestamps, No-Operation (NOP), Window scale
    TCP Option - Maximum segment size: 65495 bytes
    TCP Option - SACK permitted
    TCP Option - Timestamps: TSval 3022556121, TSecr 0
    TCP Option - No-Operation (NOP)
    TCP Option - Window scale: 7 (multiply by 128)
  [Timestamps]
0000  00 00 00 00 00 00 00 00 00 00 00 00 08 00 45 00  ....E.
0010  00 3c 85 d7 40 00 40 06 b6 e2 7f 00 00 01 7f 00  <...@.
0020  00 01 d4 7c 1f 40 3d 34 5c 32 00 00 00 00 a0 02  ...|.0=4 \2
0030  ff d7 fe 30 00 00 02 04 ff d7 04 02 08 0a b4 28  ...0.....(
0040  8b d9 00 00 00 00 01 03 03 07  ....
```

L'option Window Scale (le fameux multiplicateur) n'est négociée qu'au tout début de la conversation, pendant la phase de connexion (le *Three-way handshake*).

3-way handshake

Trame	Sens	Flags	Seq	Ack
1	54396 → 8000	SYN	0	—
2	8000 → 54396	SYN/ACK	0	1
3	54396 → 8000	ACK	1	1

Flags trame 1 : **0x002** → seul Syn: Set, tous les autres drapeaux à 0.

Fermeture de connexion TCP

La fermeture s'effectue en 4 étapes (FIN/ACK × 2) :

Trame	Sens	Flags	Seq	Ack
9	54396 → 8000	FIN/ACK	15	17
10	8000 → 54396	FIN/ACK	17	16
11	54396 → 8000	ACK	16	18

Lorsque le client envoie **Ctrl+D**, il envoie un segment **FIN/ACK** pour signaler qu'il n'a plus de données à envoyer. Le serveur répond avec son propre **FIN/ACK**, et le client confirme avec un **ACK** final. La connexion est alors complètement fermée des deux côtés.

Identifiez les segments TCP liés à la phase d'établissement de la connexion TCP ("3-way handshake") et vérifiez - par rapport au **cours** - les fanions/drapeaux/flags TCP apparaissant ainsi que les numéros de séquence et d'acquittement.

Attention, par défaut **wireshark** indique des numéros de séquence relatifs (Le premier numéro de séquence est décalé à 0). Pour voir les vrais numéros de séquence, il faut aller dans le menu préférences de **wireshark**, puis dans le protocole TCP, vous décochez la case "Relative sequence numbers". Remettez ensuite l'option pour avoir des numéros de séquence et d'acquittement relatifs (plus facile pour étudier TCP)

13. Non, les connexions ne se mélangent pas. Chaque connexion TCP est identifiée de manière unique par un **socket**, c'est-à-dire le couple (**adresse IP source, port source, adresse IP destination, port destination**). Deux onglets vers le même site utilisent des ports source différents, ce qui les distingue.

14. Capturez des trames TCP issues d'une demande de connexion vers un port non utilisé. Par exemple en ouvrant un client netcat vers un port fermé (serveur non actif sur ce port). Expliquez ce qu'il se passe.

Connexion vers un port fermé

Trame	Sens	Flags	Seq	Ack
1	57506 → 9999	SYN	0	—
2	9999 → 57506	RST, ACK	1	1

Le client envoie un **SYN** pour tenter d'établir une connexion. Comme aucun serveur n'écoute sur le port 9999, le système répond immédiatement avec un segment **RST/ACK** (Reset), ce qui signifie que la connexion est refusée. Il n'y a pas de 3-way handshake, la connexion est avortée immédiatement. **Win=0** confirme qu'aucune ressource n'est allouée côté serveur.

15. Connexion coupée — Retransmissions TCP

Une connexion SSH est établie vers la RT Box (192.31.25.1). Une fois le réseau coupé (déconnexion du câble virtuel), TCP tente de retransmettre les segments non acquittés.

On observe dans Wireshark plusieurs segments **[TCP Retransmission]** depuis 192.31.25.12 vers 192.31.25.1. Le timer de ré-émission **n'est pas fixe** : il suit un **backoff exponentiel**, c'est-à-dire que l'intervalle entre chaque retransmission double à chaque tentative (~1s, ~2s, ~4s, ~8s, ~16s...). Cela évite de saturer le réseau inutilement.

TCP insiste pendant environ **15 minutes** (par défaut sous Linux) avant d'abandonner et de signaler une erreur **Connection timed out**.

Si la machine se reconnecte avant l'abandon, TCP reprend l'échange normalement grâce aux numéros de séquence — la connexion SSH est restaurée sans interruption visible.

